

Day5 实训手册：缓存三兄弟 — 穿透、击穿、雪崩的复现与治理

一、今日任务目标

目标	说明
分清三个问题	穿透（查不存在）、击穿（热点 Key 过期）、雪崩（大量 Key 同时过期 / Redis 宕机）
会复现	每个问题都能在项目里选出一个具体接口，用 JMeter + redis-cli 亲手把问题压出来
懂方案	每个问题都掌握"预防 → 拦截 → 兜底"多层解法，并知道本课落地的是哪一层
能修复	把三味药（空对象缓存、分布式锁、随机 TTL）落进 <code>NoteService.detail</code> 一条链路里
理解分布式锁	互斥、防死锁、锁归属三要素，了解主流实现方式与选型
课堂实战	用同一套治理手段保护用户主页缓存（第八节，课内启动、课下完成）

本课的主线不再是"只讲击穿"，而是把面试里成套出现的"缓存三兄弟"逐个拆开：**先从项目里选一个真实接口把问题复现出来，再给出多层解决方案，最后把方案落进代码修复它。**三个问题最终会汇聚到同一个方法 `rebuildWithLock` 里——这正是生产代码的常态：一处缓存重建，同时挡住三种风险。

二、问题全景：缓存三兄弟

2.1 一句话定义

问题	一句话定义	危险点
缓存穿透	查一个数据库里也不存在的数据	缓存和数据库两道防线同时失效，每次请求都直达 DB
缓存击穿	单个热点 Key 过期的瞬间，海量请求同时涌入	一个 Key 的失效压垮数据库
缓存雪崩	一大片 Key 同时过期，或 Redis 整体宕机	大面积缓存失效，流量整体砸向数据库

记忆锚点：穿透是"查根本没有的"，击穿是"一个热点塌了"，雪崩是"一大片同时塌了"。三者都指向同一个后果——**请求绕过缓存直达数据库**，区别只在"塌掉的规模"和"塌掉的原因"。

2.2 术语先行（先解释，后使用）

下面两个词会从本节一直用到最后，先解释清楚，后面就不再打断：

- **回源**：缓存没命中时，“回到数据的源头”——也就是去查 MySQL（或调用下游服务）把数据取出来。“回源一次”就等于查一次数据库。回源次数越多，数据库压力越大，这正是缓存三兄弟要解决的核心；本课所有复现，本质都是在数“回源了多少次”。
- **MISS / 命中**：MISS 指缓存里没查到（未命中），需要回源；命中指缓存里查到了，直接返回、不用回源。缓存优化的一切努力，都是想让请求尽量“命中”、尽量别“回源”。

(锁相关的“互斥、双重检查、持锁 / 释放锁”等术语，留到第四节讲分布式锁时再解释。)

2.3 统一的复现基线：Day2 版 detail()

三个问题都发生在同一个方法上——笔记详情 `NoteService.detail`，对应接口 `GET /api/notes/{id}`。复现时统一回到 Day2 的“仅 Cache Aside、无任何加固”版本（源码见 `commit-files/xhs-pp-java1/day2/xhs-backend/.../service/NoteService.java`）：

```
1 // Day2 基线版 detail — 三个漏洞同时存在，是今天的“病例”
2 public NoteVO detail(Long id, Long viewerId) {
3     String key = RedisKeys.note(id);
4     NoteVO vo = (NoteVO) redisTemplate.opsForValue().get(key); // 1. 先查缓存
5     if (vo == null) { // 2. 未命中
6         vo = noteMapper.selectDetail(id); // → 直接查库
7         if (vo != null) {
8             // 固定 30 分钟 TTL；查不到 (vo=null) 不缓存 → 三大漏洞：
9             // ① 查库为空不缓存 = 穿透；② 无锁并发回源 = 击穿；③ 固定 TTL = 雪崩
10            redisTemplate.opsForValue().set(key, vo, 30, TimeUnit.MINUTES);
11        }
12    }
13    if (vo == null) {
14        return null;
15    }
16    fillStatus(Collections.singletonList(vo), viewerId);
17    return vo;
18 }
```

复现总纪律：每压一个场景前，先确认后端跑的是上面这版“未加固”的 `detail`，并把后端控制台的 SQL 日志打开——数 `selectDetail` 出现的条数，就是“回源数据库”的次数。三个问题的复现，本质都是在数这条 SQL 出现了多少次。

三、问题一：缓存穿透 — 查询“根本不存在”的数据

3.1 场景选取

项目里最容易触发穿透的接口就是笔记详情 `GET /api/notes/{id}`。正常用户点开的 `id` 都存在，但恶意攻击者会伪造海量不存在的 `id`（如 `id=-1`、`id=99999`、遍历整个 `id` 空间）并发请求。

- 接口：`GET /api/notes/99999`
- 落到方法：`NoteService.detail(99999, viewerId)` → `noteMapper.selectDetail(99999)` 返回 `null`

3.2 复现步骤（基于 Day2 基线）

1. 确认后端跑的是 Day2 未加固版 `detail`；
2. JMeter 新建线程组：200 线程、循环 1 次，全部请求 `GET /api/notes/99999`（一个数据库里不存在的 `id`）；
3. 观察后端控制台：每一次请求都打印一条 `selectDetail` 的 SQL，200 个请求 = 200 次查库；
4. 压测结束后执行 `EXISTS note:99999` → 返回 0，缓存里什么都没有。

为什么缓存永远帮不上忙？因为 Day2 版只在 `vo != null` 时才回填缓存。查不到 → 不回填 → 下次同一个 id 再来还是 MISS → 还是查库。**缓存和数据库两道防线同时失效，故称"穿透"**。生产中这就是典型的攻击手法：目的不是拿到数据，而是用不存在的 id 把你的数据库拖垮。

3.3 解决方案（三层防线）

解法一：参数校验——接口层第一道闸（最便宜）

原理：`id ≤ 0`、超过已知最大范围、格式非法的参数，在 Controller 层直接拒绝，根本不给它碰到缓存和数据库的机会。局限：只能拦"明显非法"的请求，拦不住"合法但不存在"的 id（如 `id=99999` 格式完全合法）。

解法二：空对象缓存——本课采用的方案

原理：查库为空时，不是直接返回，而是把一个"空标记"也写进缓存（短 TTL，本课 60 秒）；下次同一个 id 再来，直接命中空标记返回"不存在"，不再碰数据库。

两个细节必须注意：

- ① 空对象 TTL 必须短——如果这条数据后来真的被创建了，空对象存活期间会短暂返回"不存在"（可接受的软状态）；
- ② 读取侧要能识别空标记（本课用 `vo.getId() == null` 判断，因为空对象是 `new NoteVO()`，其 id 为 `null`）。

缺点：海量随机不存在的 id（如攻击者遍历整个 id 空间）会让空对象堆积占用内存——这时就需要解法三。

解法三：布隆过滤器（Bloom Filter）——大规模系统的主力方案

原理：启动时把**所有存在的 id** 预先放进一个巨大的位数组（用多个哈希函数把每个 id 映射到若干个位置 1）。查询时先问布隆过滤器：

```
1 请求 id=99999
2   ↓
3 布隆过滤器：对应位置有 0 → 一定不存在 → 直接返回（缓存、数据库都不用查）
4
5 请求 id=1
6   ↓
7 布隆过滤器：对应位置全是 1 → 可能存在 → 放行，继续走 Redis / MySQL
```

核心特性（面试必答）：**说"不存在"一定准，说"存在"可能误判**（不同 id 的哈希位可能重叠）——所以它适合放在缓存前面做拦截：误判的少数请求会继续往下走，但所有"一定不存在"的请求被 100% 拦下，穿透流量到不了数据库。

常见做法：Java 单机用 Guava `BloomFilter`；分布式用 Redisson 的 `RBloomFilter`（基于 Redis 位图，多实例共享）。两个局限要提前知道：标准布隆过滤器**不支持删除**（数据下架需重建，或用计数布隆过滤器）；误判率与容量需预估设置，装满了误判率会飙升。

穿透选型：本课数据量小，空对象缓存够用；大规模系统标配是"参数校验 + 布隆过滤器 + 空对象"三连招——三层各拦一部分，谁也不指望单打独斗。

3.4 修复项目代码（空对象缓存）

在重建链路里，把“查库为空”这一分支补上缓存写入。这是修复穿透的关键一步（完整方法见第六节）：

```
1 NoteVO vo = noteMapper.selectDetail(id);
2 if (vo == null) {
3     // 【防穿透】查库为空也缓存一个"空对象"，短 TTL 60 秒
4     // 读取侧用 getId()=null 识别它是空标记，直接返回"不存在"，不再查库
5     redisTemplate.opsForValue()
6         .set(RedisKeys.note(id), new NoteVO(), NULL_TTL_SECONDS,
7             TimeUnit.SECONDS);
8     return null;
9 }
```

修复后再跑 3.2 的复现：200 并发压 `id=99999`，只有第 1 个请求查库，之后 `GET note:99999` 命中空对象直接返回，`selectDetail` 从 200 条降到 1 条。

四、问题二：缓存击穿 — 单个热点 Key 过期（重点：分布式锁）

4.1 场景选取

小红书首页某条爆款笔记 `note:1` 是超级热点，缓存命中时皆大欢喜。但它的缓存总会过期——过期的一瞬间，如果恰好有成百上千个请求同时到达，它们会同时 MISS、同时冲向数据库去重建同一个 Key。

- 接口：`GET /api/notes/1`（热点笔记）
- 落到方法：`NoteService.detail(1, viewerId)`，缓存过期瞬间的并发回源

4.2 复现步骤（基于 Day2 基线）

1. 先访问一次 `GET /api/notes/1`，确认已缓存：`EXISTS note:1 = 1`；
2. 手动制造“热点过期”：`DEL note:1`；
3. JMeter 500 线程并发压 `GET /api/notes/1`（趁缓存还没重建，一起打进去）；
4. 数后端控制台的 `selectDetail` 条数：优化前约 500 条——500 个请求各查各的库，数据库瞬间被同一个 id 打了 500 次。

击穿与穿透的区别：穿透查的是“不存在”的数据（永远 MISS），击穿查的是“存在但刚好过期”的热点数据（本可命中，却在过期窗口被并发击穿）。击穿的杀伤力来自热点——一个 Key 就吸引了绝大部分流量。

4.3 理论基础：分布式锁原理与选型

击穿的根治思路是互斥重建：缓存失效时只允许 1 个线程回源查库、重建缓存，其余线程等待片刻后重读缓存。要让“只允许 1 个线程”在多实例部署下依然成立，就需要分布式锁。

锁相关术语（回源、MISS 已在 2.2 解释，这里补三个和锁直接相关的）：

- 互斥：同一时刻只允许一个线程（或进程）做某件事，其余的必须排队等待。
- 双重检查（双检）：抢到锁之后，先再读一次缓存，确认数据是不是已经被别人重建好了，避免自己白回源一次。
- 持锁 / 释放锁：成功拿到锁叫“持锁”，用完把锁删掉叫“释放锁”。

4.3.1 为什么 Java 的锁不够用了

先看 Java 自带的锁都能干什么。在单个 Java 进程（一个 JVM）里，我们有很多成熟的加锁手段，它们的共同点是：靠 JVM 内存里的一个对象或状态，来协调**同一个进程内的多个线程**。

Java 锁	典型使用场景	加锁范围
<code>synchronized</code> (方法/代码块)	最基础的互斥：同一时刻只让一个线程进入某段代码	当前 JVM 内的线程
<code>ReentrantLock</code>	需要 <code>tryLock</code> 超时、公平锁、多个条件队列等高级特性时	当前 JVM 内的线程
<code>ReadWriteLock</code>	读多写少：允许多个读线程并行，写线程独占	当前 JVM 内的线程
<code>volatile</code> / <code>Atomic*</code>	保证可见性、用 CAS 做无锁的原子计数	当前 JVM 内的线程

它们在本课为什么不够用？关键在于：这些锁的“锁状态”只存在于**某一个 JVM 进程的内存里**，进程之间互相看不见。而生产环境的后端几乎不会只跑一个实例：

1

2

3

4

5

6

7

生产部署：后端跑了 2 个实例（或多台机器），前面挂一个负载均衡
500 个并发请求 → LB {
实例A (JVM-A) : `synchronized` 只锁得住 A 自己的线程
实例B (JVM-B) : `synchronized` 只锁得住 B 自己的线程
结果：A 里放行 1 个线程回源，B 里也放行 1 个线程回源
→ 两把锁互不知情 → 同一个 `note:1` 仍然被双重回源！

一句话总结：`synchronized` 能保证“**一个进程内**只有一个线程回源”，却保证不了“**整个集群内**只有一个线程回源”。实例越多，同时回源的份数就越多——击穿的窟窿只是被缩小，并没有被堵住。要让全集群只回源一次，就必须有一把**所有实例都看得见、都要来抢**的锁，也就是把锁状态放到进程之外的公共存储（Redis）里，这就是分布式锁。

锁类型	作用范围	适用场景
<code>synchronized</code> / <code>ReentrantLock</code>	单个 JVM 进程内	单机应用（单实例部署）
分布式锁	跨进程、跨机器	多实例部署（本课）
数据库锁	跨进程，但性能差	低频场景

本课虽是单实例，但代码按多实例标准写——上线部署多实例时无需改动。这也是为什么看似“杀鸡用牛刀”也要上分布式锁：单机 `synchronized` 今天够用，明天扩容成 3 个实例就立刻失效。

4.3.2 分布式锁的三个必备要素（重点）

要素	问题	本课的解法
互斥性	同一时刻只能有一个客户端持有锁	<code>SET key NX</code> (Not eXists 才设置)
防死锁	持锁客户端崩溃，锁永远不释放	<code>EXPIRE 10秒</code> (自动过期)
锁归属	不能误删别人的锁	删除前检查 (进阶：Lua 比较值再删)

三个参数合成一条原子命令：`SET note:lock:1 1 NX EX 10`。

课堂提问：为什么不能先 `SETNX` 再 `EXPIRE`？
(两条命令之间如果崩溃，锁就永不过期 → 死锁。所以必须一条命令。)

4.3.3 主流实现方式对比（了解即可）

方案	原理	优点	缺点
Redis SETNX（本课）	SET NX EX	性能最高、已有中间件	锁超时自动过期，需业务配合
Redisson 看门狗	后台线程自动续期	解决“业务没完锁先过期”	引入额外框架（进阶题）
ZooKeeper 临时节点	会话断开节点自动删除	可靠性高、无超时问题	性能低于 Redis
数据库唯一键	插入成功=拿到锁	实现简单	性能最差，需手动清理

表格只是骨架，下面把每种方案再展开一点，帮助理解它们各自的“脾气”：

① Redis SETNX（本课采用）

直接用 Redis 的 `SET key value NX EX seconds` 一条命令抢锁：Key 不存在才写成功（NX 保证互斥），同时带上过期时间（EX 防死锁）。它最大的优势是“零额外成本”——项目本来就有 Redis，性能也是四种里最高的（纯内存操作，单实例可达十万级 QPS）。短板是锁会在固定时间后自动过期：如果业务执行时间超过了锁的 TTL，锁提前释放，就可能出现“两个线程同时持锁”。所以它需要业务方自己估准超时时间，或配合下面的 Redisson 续期。

② Redisson 看门狗（进阶）

Redisson 是 Redis 官方推荐的 Java 客户端，它在 SETNX 之上封装了一个“看门狗（watchdog）”后台线程：只要持锁的业务还没执行完，看门狗就每隔一段时间（默认锁 TTL 的 1/3）自动帮锁“续命”，从根上解决了“业务没完、锁先过期”的尴尬。它还提供了可重入锁、公平锁、读写锁、红锁（RedLock）等高级能力。代价是要额外引入一个框架依赖，对入门阶段偏重，故本课列为进阶题。

③ ZooKeeper 临时节点

利用 ZooKeeper 的“临时顺序节点”实现：客户端在 ZK 上创建一个临时节点代表持锁，会话一旦断开（进程崩溃或网络中断）节点自动删除、锁自动释放，因此天生没有“死锁”和“超时估算”问题，可靠性最高；它还能通过“监听前一个节点”实现公平排队。缺点是 ZooKeeper 为强一致（ZAB 协议）而设计，写入要过半节点确认，性能明显低于 Redis，且需要额外维护一套 ZK 集群。

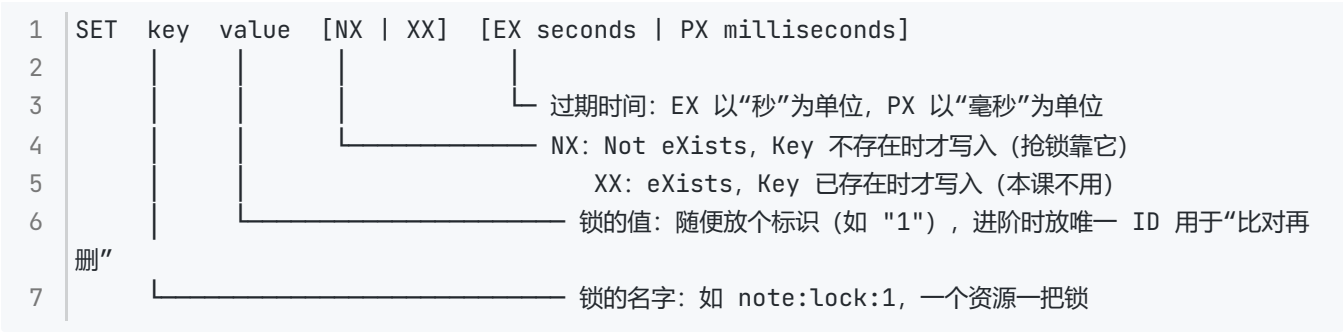
④ 数据库唯一键

最朴素的思路：建一张锁表，对某个字段加唯一索引，谁能 `INSERT` 成功谁就拿到锁，用完 `DELETE` 释放。优点是无需任何中间件、实现直白。缺点也最多：数据库操作性能最差；并发抢锁时大量 `INSERT` 失败会带来无谓开销；持锁进程崩溃后锁不会自动释放，需要额外的超时清理逻辑。一般只用于对性能不敏感的低频场景。

选型结论：缓存重建场景容忍极小概率的双重回源，追求高性能，且项目已有 Redis → 选 Redis SETNX 方案；若后续业务耗时不可控，再平滑升级到 Redisson 看门狗。

4.3.4 动手：亲手拿一次分布式锁（上课必做）

先把 `SET` 命令的语法拆开看清楚。本课抢锁用的是 Redis `SET` 命令带可选参数的形式：



各参数逐个说明：

参数	作用	缺了会怎样
key	锁的唯一名字，代表“被保护的资源”（如 <code>note:lock:1</code> 保护 1 号笔记的重建）	—
value	锁的持有者标识。入门放 <code>"1"</code> 即可；生产建议放“请求唯一 ID”，释放锁时先比对比值再删，防止误删别人的锁	无法区分锁是谁加的，容易误删
NX	互斥的核心 ：只有 Key 不存在时才写成功并返回 <code>OK</code> ；Key 已存在则什么都不做、返回 <code>nil</code>	不加 NX 就成了普通覆盖写，人人都能“抢到”，锁形同虚设
EX seconds	防死锁的核心 ：给锁设一个自动过期时间（秒），持锁进程崩溃后锁会自己消失	不加过期时间，持锁者一崩溃锁就永久残留 → 死锁

关键认知：NX 和 EX 必须写在同一条 SET 命令里，Redis 才会原子地“要么都成功、要么都不做”。这正是 4.3.2 那个提问“为什么不能先 SETNX 再 EXPIRE”的答案——分成两条命令，中间一旦崩溃，就只加了锁却没设过期时间，锁永远不释放。

进入容器：`docker exec -it xhs-redis redis-cli:`

```
1 # ① 抢锁：返回 OK = 抢到；再执行一次返回 nil = 锁被别人持有（互斥生效）
2 SET demo:lock "1" NX EX 10
3 SET demo:lock "1" NX EX 10
4
5 # ② 查看锁剩余时间（防死锁的体现，应看到 10 以内的正整数在递减）
6 TTL demo:lock
7
8 # ③ 释放锁（生产上应先比对比 value 再删，防止误删）
9 DEL demo:lock
```

课堂讨论：如果业务执行了 15 秒，锁 10 秒就过期了会发生什么？
(第二个客户端拿到锁 → 双重回源/误删锁。应对：估准超时、Lua 比对删除、或引入 Redisson 看门狗自动续期。)

4.4 解决方案

方案一：互斥重建（分布式锁，★本课落地）

缓存 MISS 时先抢锁，抢到的线程负责查库重建，没抢到的线程等待后重读缓存。保证同一时刻只有一个线程回源，把 500 次查库压成 1 次。代价：没抢到锁的线程要短暂等待，接口延迟略增；需要处理锁超时、误删等边界。

方案二：热点数据逻辑过期（呼应 Day7）

不给缓存设物理 TTL，而是在 Value 里存一个"逻辑过期时间"字段；请求发现逻辑过期后，返回旧数据的同时异步起一个线程去刷新。缓存"永不物理失效"，从根上没有"过期瞬间"。代价：数据有秒级延迟，实现更复杂（Day7 热榜会用到这个思路）。

4.5 修复项目代码（分布式锁互斥重建）

互斥重建流程：

```
1 请求 → Redis MISS
2  ↓
3  SET note:lock:{id} 1 NX EX 10
4  ┌─ 抢到锁 → 再次检查缓存（双检）→ 查MySQL → 回填缓存 → 释放锁
5  └─ 没抢到 → 短暂等待 → 重读缓存（此时大概率已重建完成）
```

对应代码骨架（完整方法见第六节）：

```
1 private NoteVO rebuildWithLock(Long id) {
2     String lockKey = RedisKeys.noteLock(id);
3     // 【防击穿】抢锁：只有抢到的线程能回源，其余线程等待重读
4     Boolean locked = redisTemplate.opsForValue().setIfAbsent(lockKey, "1",
5         LOCK_TTL_SECONDS, TimeUnit.SECONDS);
6
7     if (!Boolean.TRUE.equals(locked)) {
8         // 没抢到锁：等一下再读缓存，此时大概率已被别人重建好
9         try {
10             Thread.sleep(50);
11         } catch (InterruptedException e) {
12             Thread.currentThread().interrupt();
13         }
14         NoteVO vo = getFromCache(id);
15         return vo != null ? vo : noteMapper.selectDetail(id); // 极端兜底
16     }
17
18     try {
19         NoteVO vo = getFromCache(id); // 双重检查：等锁期间可能已被重建
20         if (vo != null) {
21             return vo;
22         }
23         vo = noteMapper.selectDetail(id); // 真正回源（全场仅此一次）
24         // ... 回填缓存（含防穿透、防雪崩，见第六节）
25         return vo;
26     } finally {
27         redisTemplate.delete(lockKey); // 释放锁（进阶：Lua 比对值再删）
28     }
29 }
```

修复后再跑 4.2 的复现：DEL note:1 + 500 并发，selectDetail 从约 500 条降到约 1 条，压测期间错误率为 0（没抢到锁的请求等待后读到了新缓存）。

五、问题三：缓存雪崩 — 一大片缓存同时失效

5.1 场景选取

雪崩在本项目里对应两种真实场景：

- **成因 A (集中过期)**：系统启动时批量预热首页、榜单、热门笔记等一批 Key，如果统一设了 30 分钟 TTL，半小时后它们会在同一时刻集体阵亡，全部流量瞬间砸向数据库。
- **成因 B (Redis 宕机)**：Redis 单节点整体挂掉，所有缓存瞬间消失，全部读请求直达数据库。

5.2 复现步骤 (基于 Day2 基线)

复现成因 A：

1. 用 redis-cli 批量预热几个 Key，故意设**相同**的 TTL：

```
1 SET note:1 "... " EX 1800
2 SET note:2 "... " EX 1800
3 SET note:3 "... " EX 1800
```

2. 执行 `TTL note:1`、`TTL note:2`、`TTL note:3` → **三个值几乎完全相同**；
3. 结论：30 分钟后它们会在同一秒集体过期，届时对这三个笔记的所有请求会同时回源——这就是雪崩的雏形。

成因 B (Redis 宕机) 在课堂上用语言描述即可：`docker stop xhs-redis` 会让所有缓存瞬间消失。真正防御它靠的是高可用部署，而不是某段业务代码，所以本课的落地修复聚焦成因 A。

5.3 解决方案

雪崩有两类成因，解法完全不同，先分清再对症下药：

针对成因 A：大量 Key 在同一时刻集中过期

- **随机 TTL (★本课落地)**：过期时间 = 基础值 + 随机抖动，把"死亡时刻"打散到一个时间窗内，避免集体阵亡。本课写法：`30分钟 + random(0~5分钟)`。
- **热点数据逻辑过期 / 定时刷新**：对榜单、首页这类热点，后台任务在过期前主动刷新，缓存"永生"——代价是数据有秒级延迟（与 4.4 方案二同源，Day7 热榜会用到）。

针对成因 B：Redis 整体宕机

- **高可用部署**：主从 + 哨兵或 Cluster (Day2 手册 3.5 节学的部署形态在这里派上用场)——单节点挂了自动切换，从根上降低"整体宕机"概率；
- **多级缓存**：应用进程内再加一层本地缓存 (Caffeine/Guava Cache, 容量小、纳秒级)，Redis 挂了本地缓存还能顶一阵，至少热点数据不死；
- **限流降级**：数据库眼看要被打死时，先保数据库不死——接口限流挡住超额流量，降级返回兜底数据。这正是 Day8 要落地的内容，也是雪崩的最后一道保险丝。

5.4 修复项目代码（随机 TTL 抖动）

在回填缓存时，把固定 TTL 改成“基础值 + 随机抖动”（完整方法见第六节）：

```
1 // 【防雪崩】TTL = 30 分钟基础值 + 0~5 分钟随机抖动，把过期时刻打散
2 long ttl = BASE_TTL_SECONDS + ThreadLocalRandom.current().nextLong(JITTER_SECONDS);
3 redisTemplate.opsForValue().set(RedisKeys.note(id), vo, ttl, TimeUnit.SECONDS);
```

修复后执行 TTL note:1、TTL note:2、TTL note:3 → 三个过期时间互不相同，集体阵亡被拆解成错峰过期。

六、三味药合一：完整的 rebuildWithLock

三个问题的修复最终都落在同一个方法里——这正是生产代码的常态。下面是 `code/day5/service/NoteService.java` 的完整实现，三处关键修复已用【防穿透】【防击穿】【防雪崩】标注：

```
1 /** 基础缓存时长（秒）：30分钟 */
2 private static final long BASE_TTL_SECONDS = 1800;
3 /** 随机抖动上限（秒）：防雪崩 */
4 private static final long JITTER_SECONDS = 300;
5 /** 空对象缓存时长（秒）：防穿透 */
6 private static final long NULL_TTL_SECONDS = 60;
7 /** 重建锁超时（秒）：防持锁线程崩溃导致死锁 */
8 private static final long LOCK_TTL_SECONDS = 10;
9
10 /** ★ Day5 改造：缓存 → 未命中走分布式锁互斥重建 */
11 public NoteVO detail(Long id, Long viewerId) {
12     NoteVO vo = getFromCache(id);
13     if (vo == null) {
14         vo = rebuildWithLock(id);          // 未命中 → 互斥重建
15     }
16     // 空对象缓存 (getId()=null) 表示笔记不存在
17     if (vo == null || vo.getId() == null) {
18         return null;
19     }
20     fillStatus(Collections.singletonList(vo), viewerId);
21     mergeCounts(Collections.singletonList(vo));
22     return vo;
23 }
24
25 /** 读缓存 */
26 private NoteVO getFromCache(Long id) {
27     return (NoteVO) redisTemplate.opsForValue().get(RedisKeys.note(id));
28 }
29
30 /**
31  * ★ 分布式锁互斥重建 —— 一个方法同时挡住三兄弟
32  * 抢到锁 → 双重检查 → 查库 → 回填（空对象/随机TTL）→ 释放锁
33  * 没抢到 → 短暂等待后重读缓存（此时大概率已重建完成）
34  */
35 private NoteVO rebuildWithLock(Long id) {
36     String lockKey = RedisKeys.noteLock(id);
```

```

37 // 【防击穿】抢锁：只有抢到的线程回源，其余线程等待重读
38 Boolean locked = redisTemplate.opsForValue()
39     .setIfAbsent(lockKey, "1", LOCK_TTL_SECONDS, TimeUnit.SECONDS);
40
41 if (!Boolean.TRUE.equals(locked)) {
42     try {
43         Thread.sleep(50);
44     } catch (InterruptedException e) {
45         Thread.currentThread().interrupt();
46     }
47     NoteVO vo = getFromCache(id);
48     // 极端情况下仍为空，直接回源（由教师讲解此处取舍）
49     return vo != null ? vo : noteMapper.selectDetail(id);
50 }
51
52 try {
53     // 双重检查：可能在等锁期间缓存已被别人重建
54     NoteVO vo = getFromCache(id);
55     if (vo != null) {
56         return vo;
57     }
58     vo = noteMapper.selectDetail(id);
59     if (vo == null) {
60         // 【防穿透】查库为空也缓存一个空对象，短 TTL，挡住反复查不存在 ID
61         redisTemplate.opsForValue()
62             .set(RedisKeys.note(id), new NoteVO(), NULL_TTL_SECONDS,
63                 TimeUnit.SECONDS);
64         return null;
65     }
66     // 【防雪崩】TTL 加随机抖动，避免大量 Key 同时过期
67     long ttl = BASE_TTL_SECONDS +
68         ThreadLocalRandom.current().nextLong(JITTER_SECONDS);
69     redisTemplate.opsForValue().set(RedisKeys.note(id), vo, ttl,
70         TimeUnit.SECONDS);
71     return vo;
72 } finally {
73     // 简化版释放：生产环境建议用 Lua "比较值再删除" 防止误删别人的锁
74     redisTemplate.delete(lockKey);
75 }

```

6.1 关键细节决策表

细节	决策	理由	对应问题
抢锁 <code>setIfAbsent</code>	NX + EX 10 秒	只放一个线程回源，且防死锁	击穿
双重检查	拿到锁后再读一次缓存	避免等锁期间别人已重建导致的重复回源	击穿
空对象	查库为空也缓存 60 秒	挡住反复查不存在 id	穿透
随机 TTL	30 分钟 + 0~5 分钟抖动	打散过期时刻	雪崩

6.2 三兄弟防线总表（面试可直接背）

问题	第一道防线	主力方案	兜底
穿透	参数校验	布隆过滤器	空对象缓存（★本课）
击穿	热点数据逻辑过期	分布式锁互斥重建（★本课）	双重检查减回源
雪崩	随机 TTL（★本课）	高可用部署 + 多级缓存	限流降级（Day8）

记忆方法：每个问题都是"预防 → 拦截 → 兜底"三层，本课落地的是每个问题的核心那一层，其余两层在 Day2/Day7/Day8 都有呼应。

面试常问："缓存穿透、击穿、雪崩分别怎么解决？"——先给定义（各一句话），再按三层防线答，最后补一句"我在实训项目里用分布式锁 + 空对象 + 随机 TTL 落地过，压测验证 500 并发回源仅 1 次"。

七、验证与压测

7.1 三场景验收步骤

场景	步骤	优化前	优化后（达标）
穿透	200 并发压 GET /api/notes/99999	每次查库（~200 条 SQL）	仅第 1 次查库，之后空对象命中（1 条 SQL）
击穿	DEL note:1 后 500 并发压 GET /api/notes/1	~500 条 selectDetail	约 1 条，错误率 0
雪崩	TTL note:1、TTL note:2、TTL note:3	三个值几乎相同	三个过期时间互不相同

7.2 结果记录表

场景	优化前回源次数	优化后回源次数	是否达标
穿透（200 并发）	~200	1	
击穿（500 并发）	~500	~1	
雪崩	固定 30 分钟	随机抖动	

八、课堂实战：保护用户主页缓存（自己动手）

今天老师带做的是笔记详情缓存的治理，现在轮到你把同一套治理手段用到另一个热点接口。用户在小红书点开某个博主主页是高频操作，GET /api/users/{id} 目前每次都查库。

- 需求：给 UserService.userInfo(id, viewerId) 加上"缓存 + 分布式锁重建"，三味解药一样不少：
- 缓存 Key: user:{id}，Value 存 UserVO 的 JSON；
 - 互斥重建：未命中时抢锁 user:lock:{id}（NX + EX 10 秒），抢到才查库，其他线程等待重读；

- **空对象缓存**: 查不到该用户时缓存一个空对象 60 秒, 防止穿透;
- **随机 TTL**: 基础 30 分钟 + 0~5 分钟随机抖动, 防雪崩。

实现步骤:

1. `RedisKeys.java` 新增 `user(id)` (返回 `"user:" + id`) 与 `userLock(id)` (返回 `"user:lock:" + id`);
2. 完全参照 `NoteService.detail` 的 `rebuildWithLock` 三步结构写一个 `loadUserWithLock`: 拿锁 → 双检缓存 → 查库回填 (含空对象) → finally 释放锁;
3. 随机 TTL 用 `ThreadLocalRandom.current().nextLong(0, 300)` 秒 (或 `nextInt(0, 5)` 分钟);
4. 空对象识别用 `vo.getId() == null` (`UserVO` 没有 `nullFlag` 字段, 别自造 API), 验证三个场景。

关键代码提示 (骨架, 细节参照 `NoteService`):

```
1 public UserVO userInfo(Long id, Long viewerId) {
2     String key = RedisKeys.user(id);
3     UserVO vo = (UserVO) redisTemplate.opsForValue().get(key);
4     if (vo != null) {
5         return vo.getId() == null ? null : vo;    // 命中空对象 = 用户不存在
6     }
7     Boolean locked = redisTemplate.opsForValue()
8         .setIfAbsent(RedisKeys.userLock(id), "1", 10, TimeUnit.SECONDS);
9     if (Boolean.TRUE.equals(locked)) {
10         try {
11             vo = (UserVO) redisTemplate.opsForValue().get(key);    // 双重检查
12             if (vo != null) {
13                 return vo.getId() == null ? null : vo;
14             }
15             User user = userMapper.selectById(id);
16             if (user == null) {
17                 // 【防穿透】查不到也缓存空对象 60 秒
18                 redisTemplate.opsForValue()
19                     .set(key, new UserVO(), 60, TimeUnit.SECONDS);
20                 return null;
21             }
22             vo = new UserVO();
23             BeanUtils.copyProperties(user, vo);
24             // ... 填充 noteCount/followCount/fansCount (参照原 userInfo)
25             // 【防雪崩】回填 TTL 加随机抖动
26             long ttl = 1800 + ThreadLocalRandom.current().nextLong(300);
27             redisTemplate.opsForValue().set(key, vo, ttl, TimeUnit.SECONDS);
28             return vo;
29         } finally {
30             redisTemplate.delete(RedisKeys.userLock(id));
31         }
32     }
33     // 没抢到锁: 等待 + 重读 (注意 InterruptedException 处理)
34     try {
35         Thread.sleep(50);
36     } catch (InterruptedException e) {
37         Thread.currentThread().interrupt();
38     }
```

```
39     return userInfo(id, viewerId);
40 }
```

验收标准：

DEL user:1 后 500 并发压用户主页，回源 SQL 约 1 条；

压测 **GET /api/users/99999**（不存在），仅第 1 次查库，之后空对象命中；

TTL user:1 与 **TTL user:2** 的过期时间不相同。

选做挑战：如果用户改了昵称，主页缓存怎么失效？（提示：写操作删缓存，和 Day2 第八节的评论列表同一个套路）

九、思考题

1. 没抢到锁的线程为什么要"等待 + 重读"而不是直接查库？
2. 释放锁为什么最好在 Lua 里"比较值再删除"？（防误删别人的锁）
3. 如果是"热点榜单"这种由计算得出的数据，击穿保护要加在哪一层？
4. 三兄弟的防线各有三层（预防/拦截/兜底），本课只落地了每个问题的核心一层。请为"穿透"和"雪崩"各补出另外两层，并说明它们分别在哪一天的课程里出现过。